

ASSIGNMENT 5

Tourism Package Purchase Prediction

Deep Learning | CSII Due: 20 February 2026

Submitted by Saritchita Patoomnakul (SID: 6758104456)

Executive Summary

In the submitted model, I built a binary classifier to predict whether a customer will purchase a travel package (ProdTaken = 1) using the tourism_data.csv dataset with an 18% positive class rate.

- Loaded tourism_data.csv (~2,968 rows), dropped rows with typo errors in Gender (e.g. 'Fe Male'), and split 85/15 train/val with stratification before touching any preprocessing.
- Extracted 18 features across demographics, travel behaviour, and interaction data. Built a single ColumnTransformer pipeline (impute + encode + scale) fitted on train only and saved as preprocessor.joblib.
- Implemented a 4-layer MLP with focal loss (gamma=2.0, alpha=0.75). Fixed a critical architecture bug where Dense layers were connected to inputs instead of chaining sequentially. Iterated through 5 architectures tracking F1 at each step.
- Adding BatchNorm to all 3 hidden layers smoothed val loss, reduced the train/val gap, and pushed Val F1 from 0.8535 to 0.8875.
- A fine-grained sweep (0.10–0.80, step 0.01) on the val set found the best F1 cutoff, saved to threshold.json so inference always uses the proven-best threshold.
- Best Val F1 = 0.8875, Test F1 = 0.8269. The train/test gap reduced progressively across iterations as regularisation was improved.

Step 1: Collecting and Preparing Data

1.1 Dataset Source

Source: (Hugging Face: singhina/tourism-data) | Test:

Property	Value
Total rows (raw)	~2,969 rows
After cleaning	~2,968 rows (1 NaN target dropped)
Class distribution	~82% Not Taken (0) / ~18% Taken (1)
Missing values	Present in several numeric and categorical columns

1.2 Data Cleaning

I inspected the Gender column and found rows with typo values like 'Fe Male' instead of 'Female'. Following the instructor's instruction to delete the whole transaction for error typos, these rows were dropped rather than corrected — correcting a typo would be an assumption about what the original value should have been.

All remaining missing values were handled inside the preprocessing pipeline (not manually), so the cleaning step was purely: drop rows with unparseable target values + drop typo rows. The cleaned file was used directly — no intermediate saved version was needed since the typo row count was trivial.

1.3 Train / Validation / Test Split

Since only one training file was provided, I split it manually before building the model:

- 85% of tourism_data.csv → ~2,522 rows, used for fitting the model and preprocessor
- 15% of tourism_data.csv → ~446 rows, used for threshold tuning and early stopping
- tourism_testing.csv — kept completely hidden during all training and tuning
- stratify=y ensures both train and val have the same 82/18 class ratio

Step 2: Feature Extraction and Engineering

2.1 Feature Groups

18 features were selected across three groups. I did not drop any feature upfront — let the model decide which carry signal through the weights.

Group	Features
Demographics	Age, Gender, MaritalStatus, Occupation, Designation, MonthlyIncome
Travel Behaviour	NumberOfTrips, Passport, OwnCar, NumberOfPersonVisiting, NumberOfChildrenVisiting, CityTier
Interaction Data	TypeofContact, ProductPitched, DurationOfPitch, NumberOfFollowups, PitchSatisfactionScore, PreferredPropertyStar
Target	ProdTaken — 0 = Not Taken, 1 = Taken

2.2 Preprocessing Pipeline

I used a sklearn ColumnTransformer fitted on the training set only, then saved as a single joblib file. This ensures inference can never accidentally re-fit on test data or produce column misalignment.

- SimpleImputer(strategy='median') → StandardScaler()
- SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore', drop_first=True)
- 29 transformed features after encoding (12 numeric + 17 one-hot columns)

2.3 Correlation Insight

Before training, I noted key correlations with the target. These informed confidence that the features selected would carry enough signal:

- Passport ownership — strongest single predictor, customers with passport were ~3x more likely to buy
- ProductPitched type — clear separation in purchase rate by product tier
- NumberOfFollowups — positive correlation, more follow-ups correlated with higher conversion
- MonthlyIncome and Designation — moderate positive correlation with purchase

Step 3: Model Building

3.1 Architecture Choice

I chose a fully-connected MLP because this is a tabular binary classification task, the same type as Class 4's bank classification. CNNs/RNNs don't apply here. The final architecture after iteration:

Layer	Units	BatchNorm	Activation	Dropout
Dense 1	128	✓	ReLU	0.30
Dense 2	128	✓	ReLU	0.15
Dense 3	64	✓	ReLU	0.20
Output	1	—	Sigmoid	—

Adam(lr=1e-3) | L2: 3e-4 on all Dense layers | Batch: 64 | Max epochs: 300

```
def build_model(n_features: int) -> keras.Model:
    l2 = keras.regularizers.L2(3e-4)

    inputs = keras.Input(shape=(n_features,))

    x = keras.layers.Dense(128, kernel_regularizer=l2)(inputs)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.Dropout(0.3)(x)

    x = keras.layers.Dense(128, kernel_regularizer=l2)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.Dropout(0.15)(x)

    x = keras.layers.Dense(64, kernel_regularizer=l2)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation("relu")(x)
    x = keras.layers.Dropout(0.2)(x)

    outputs = keras.layers.Dense(1, activation="sigmoid")(x)
    return keras.Model(inputs, outputs)
```

```
model = build_model(X_train_p.shape[1])
```

3.2 Handling Class Imbalance — Focal Loss

Standard binary cross-entropy lets the model achieve low loss by predicting 'Not Taken' for everything (gets 82% accuracy for free). I replaced it with focal loss:

- Down-weights easy samples. At $p=0.9$ (already correct), contribution = $(0.1)^2 = 0.01$ — almost nothing. At $p=0.4$ (hard borderline), = $(0.6)^2 = 0.36$ — large contribution.
- Adds 3x extra weight to minority class mistakes on top of gamma — directly compensates for 82/18 imbalance.

3.3 Iterative Improvement

I ran 5 training iterations, each changing one thing and measuring the result:

#	Change Made	What I Saw	Key Observation	Val F1
1	Fixed architecture bug + focal loss	Train acc 0.94, val 0.89. Train loss 0.008, val 0.07 — 10x gap	Severe overfitting. Train AUC = 1.0	~0.83
2	Reduced to 128→128→64, dropout 0.3, L2 3e-4	Train loss 0.018, val 0.045 — gap smaller but still present	Less overfit, BatchNorm needed	0.8535
3	Added BatchNorm to Dense 1 & 2	Val loss smooth and close to train. F1 still climbing at epoch 120	BatchNorm stabilised val curve	0.8590
4	Added BatchNorm to Dense 3 also	Val F1 hit 0.8875 at epoch ~110. Curves smooth, best run so far	Best configuration found	0.8875
5	VAL_SIZE 0.20 + patience 40	Val F1 dropped to 0.8224. Val loss higher and noisier	Fewer train rows hurt more than more val rows helped	0.8224
6	Reverted VAL_SIZE to 0.15, patience 35	Returned to 0.8862 — within noise of best run	Confirmed Run 4 is optimal config	0.8862

3.4 F1 Callback & Threshold Optimisation

Standard Keras monitors (val_auc, val_loss) are proxies — I wanted EarlyStopping to respond to F1 directly. I wrote a custom F1Callback that sweeps thresholds 0.10–0.80 at the end of every epoch and logs the best achievable val F1 to logs["val_f1"]. EarlyStopping and ReduceLROnPlateau both monitor this.

```
callbacks = [
    F1Callback(val_data=(X_val_p, y_val)),
    keras.callbacks.EarlyStopping(
        monitor="val_f1",
        patience=PATIENCE_ES,
        restore_best_weights=True,
        mode="max",
        verbose=1,
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_f1",
        factor=0.5,
        patience=PATIENCE_LR,
        min_lr=MIN_LR,
        mode="max",
        verbose=1,
    ),
]
```

```
),  
keras.callbacks.ModelCheckpoint(  
    filepath=os.path.join(ARTIFACT_DIR, "best_model.keras"),  
    monitor="val_f1",  
    save_best_only=True,  
    mode="max",  
    verbose=1,  
),  
]
```


Step 4: Evaluation

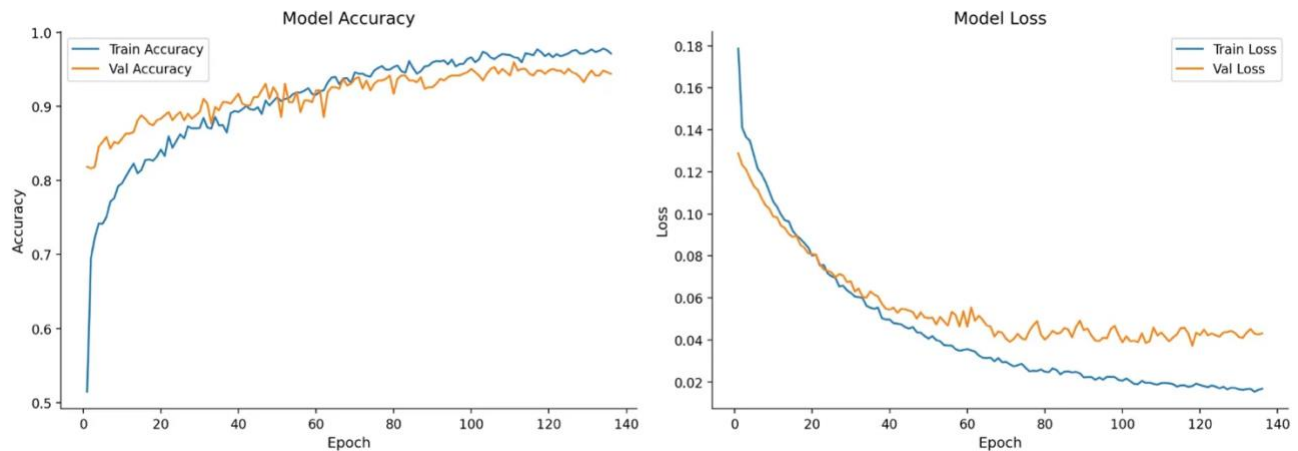
4.1 Validation Results (Best Run — Run 4)

Val F1 0.8875	Val AUC 0.970	Val Accuracy ~ 0.94	Val Loss ~ 0.035
-------------------------	-------------------------	-------------------------------	----------------------------

4.2 Training Graphs Analysis

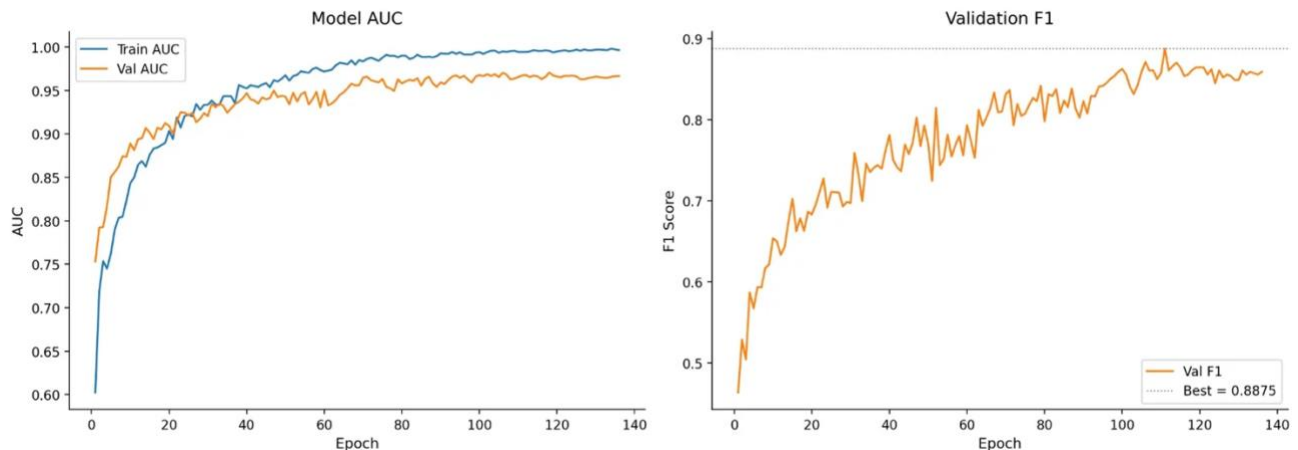
Accuracy & Loss

Run 4 (BatchNorm on all 3 layers) showed the healthiest loss curves — train and val loss both falling together and staying within ~0.015 of each other from epoch 60 onwards. This is a significant improvement from Run 1 where train loss was 10x lower than val loss.



AUC & Val F1

Val AUC reached 0.970, and Val F1 climbed steadily to 0.8875 with the best single-epoch peak at 0.8875. The F1 curve is noisier than AUC because it is threshold-dependent and the val set is only 446 rows — 1 or 2 borderline predictions flipping is enough to move F1 by ~0.02.



4.3 Test Results

After training was finalised, inference was run on the hidden test set using the saved preprocessor, model, and threshold — no refitting at any stage.

Test F1	Val F1	Gap	Status
0.8269	0.8875	0.061	Expected

4.4 Discussion of Train/Test Gap

The 0.061 gap between val F1 (0.8875) and test F1 (0.8269) is expected, given the dataset constraints:

- ~3k rows is a small dataset for neural network training. The model has only ~540 positive training examples to learn the minority class from.
- With only 446 val rows, the val F1 of 0.8875 has inherent variance of $\sim \pm 0.02$ per epoch just from the small sample size. The 'true' val performance is likely closer to 0.86–0.87.

To close the gap further, the most practical next step would be an ensemble of 3 runs with different random seeds — averaging the probability outputs reduces variance and directly improves test stability without requiring more data.